

Tokenization Evaluation and Character Normalization

Comparative preprocessing experiments on informal Twitter/X text

Document	Assignment Report			
Course	Information Retrieval			
Dataset	Tesla Twitter Corpus (Tesla.csv)			
Scope	Task 1	-	Tokenization	Evaluation
	Task 2 - Repeated Character Normalization			
Repository	AbhaySingh0303/information-retrieval-assignment			

Technical submission based on executed scripts and saved repository outputs.

Abstract

This report documents two preprocessing experiments for Information Retrieval on the Tesla Twitter/X corpus. Task 1 evaluates three tokenizers - NLTK TweetTokenizer, spaCy, and NLTK word_tokenize - using exact preservation of hashtags, mentions, URLs, and emojis as the main criterion. Task 2 studies repeated-character normalization on a separate 100-post subset. The normalization code protects URLs, hashtags, and mentions, then uses candidate reductions scored with word frequency. The saved outputs also expose an important limitation: cashtags are not explicitly protected and can be altered by the repetition rule.

1. Introduction

Informal social-media text differs from conventional prose because many punctuation-like forms carry meaning. A hashtag can identify a topic, an @mention can identify a user, and a URL is meaningful only when kept intact. Repeated characters add another source of variation: forms such as “Goood”, “Brooooooooo”, and “waaaaay” may express emphasis but also create separate vocabulary entries during indexing.

1.1 Objectives

- Compare a tweet-specific tokenizer with two general-purpose alternatives on the same 50 posts.
- Measure exact preservation of hashtags, mentions, URLs, and emojis, together with token counts.
- Detect alphabetic runs of three or more repeated characters in a disjoint 100-post subset.
- Normalize elongated forms using lexical frequency while keeping protected social-media entities unchanged.
- Inspect both successful corrections and failure cases produced by the implemented pipeline.

1.2 Data and Reproducibility

The source file is Tesla.csv. Task 1 uses a deliberately feature-rich 50-post subset: the selection script prioritizes posts containing hashtags, mentions, URLs, emojis, and common slang before filling any remaining places. Task 2 removes the Task 1 texts, filters for posts matching the repeated-letter pattern, and samples 100 posts with `random_state = 42`. Source code and generated outputs are available in the repository: github.com/AbhaySingh0303/information-retrieval-assignment

2. Task 1 - Experimental Design

2.1 Tokenizers Compared

Tokenizer	Purpose in the experiment	Configuration
NLTK TweetTokenizer	Twitter-aware rule-based tokenizer	preserve_case=True; reduce_len=False; strip_handles=False
spaCy (en_core_web_sm)	General-purpose NLP tokenizer	Default English pipeline tokenization
NLTK word_tokenize	Conventional word-tokenization baseline	NLTK word_tokenize()

Table 1. Tokenizers used in the comparative experiment.

2.2 Ground Truth Construction

Ground-truth entities are extracted from the original text before any tokenizer is applied. The same extracted strings are then searched for as complete tokens in each tokenizer output.

Element	Extraction rule	Example
Hashtag	(?<!w)#[A-Za-z0-9_]+	#Tesla
Mention	(?<!w)@[A-Za-z0-9_]+	@elonmusk
URL	https?://[^\s]+ www\.[^\s]+	https://example.com
Emoji	emoji.emoji_list(text)	Unicode emoji sequence

Table 2. Ground-truth extraction used by ground_truth.py.

2.3 Evaluation Criterion

A target counts as preserved only when the complete ground-truth string occurs as one token. For example, #Tesla is correct as ["#Tesla"] but not as ["#", "Tesla"]. The same all-or-nothing rule is used for mentions, URLs, and emoji sequences.

2.4 Processing Sequence

1. Select the 50-post Task 1 subset.
2. Extract hashtags, mentions, URLs, and emojis to create task1_ground_truth.csv.
3. Run all three tokenizers on the identical posts.
4. Compare each tokenizer output against the exact ground truth.
5. Write tokenizer_results.csv, evaluation_summary.csv, and the detailed error analysis.

3. Task 1 - Results

Tokenizer	Hashtag %	Mention %	URL %	Emoji %	Tokens	Avg./post
TweetTokenizer	100.00	100.00	100.00	95.35	1,814	36.28
spaCy	0.00	97.78	100.00	95.35	2,013	40.26
word_tokenize	0.00	0.00	0.00	34.88	2,151	43.02

Table 3. Preservation rates and token counts from evaluation_summary.csv.

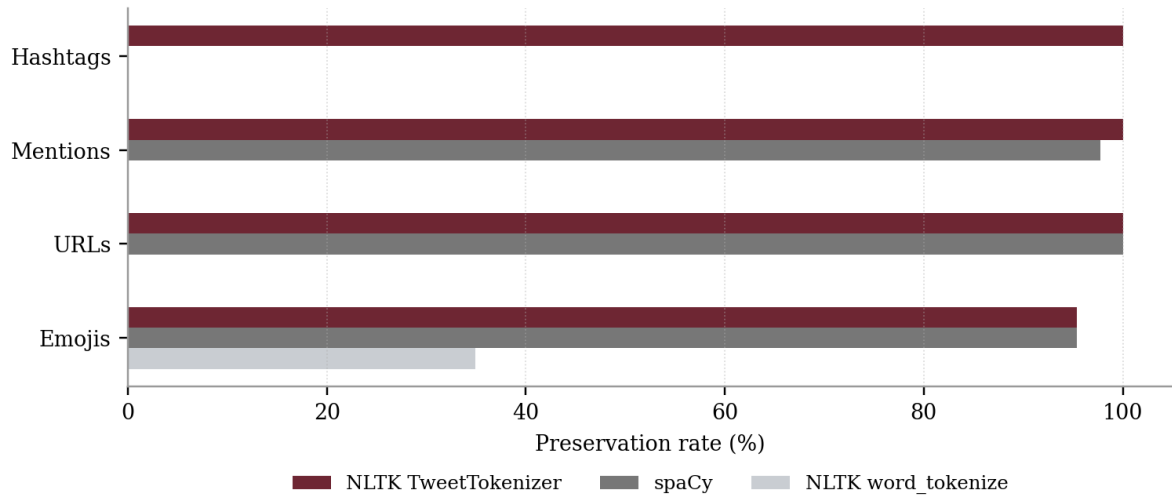


Figure 1. Exact preservation of social-media elements.

3.1 Interpretation

Hashtags: TweetTokenizer preserved every hashtag. spaCy and word_tokenize split the marker from the following text, producing a 0% exact-preservation score.

Mentions and URLs: TweetTokenizer preserved both categories completely. spaCy preserved all URLs and 97.78% of mentions, while word_tokenize preserved neither category under the strict single-token criterion.

Emojis: TweetTokenizer and spaCy both reached 95.35%. word_tokenize reached 34.88%, showing much weaker handling of emoji sequences.

4. Tokenization Findings

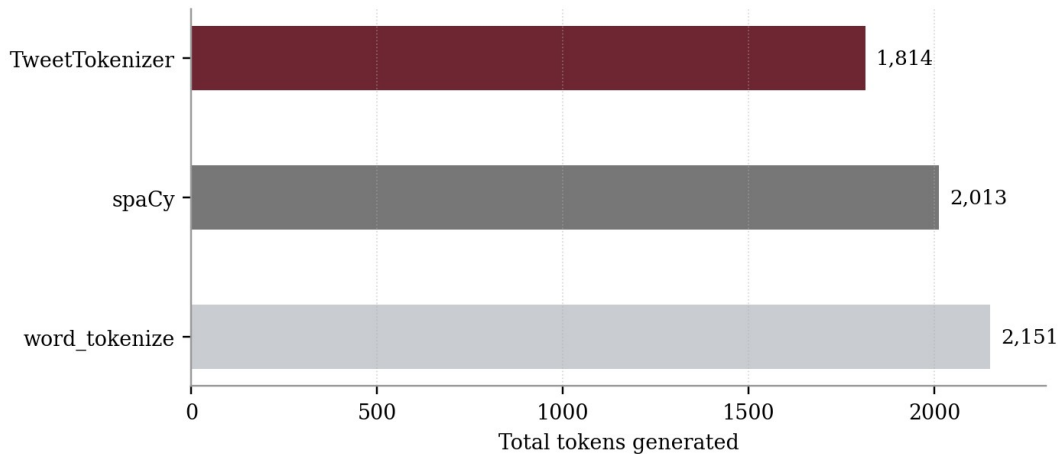


Figure 2. Total number of tokens generated by each tokenizer.

4.1 Token Count and Fragmentation

NLTK word_tokenize generated 2,151 tokens, compared with 2,013 for spaCy and 1,814 for TweetTokenizer. The higher count is not evidence of higher quality in this experiment. It largely reflects additional splitting of hashtags, mentions, URLs, and other structures that the evaluation treats as meaningful units.

4.2 Error-post Counts

Tokenizer	Hashtag errors	Mention errors	URL errors	Emoji errors
TweetTokenizer	0	0	0	2
spaCy	23	3	0	2
word_tokenize	23	44	24	13

Table 4. Number of posts containing at least one preservation error.

4.3 Task 1 Conclusion

For the features tested here, NLTK TweetTokenizer is the most suitable of the three tokenizers. It preserves Twitter-native structures with the least fragmentation and also produces the smallest token count. spaCy remains a strong alternative when a broader NLP pipeline is required, but its default tokenization is not hashtag-aware. The conventional word_tokenize baseline is the least suitable for this informal social-media setting.

5. Task 2 - Repeated Character Normalization

5.1 Subset Selection

Task 2 uses a separate 100-post subset. The selection script first removes all Task 1 texts, removes duplicate tweets, keeps posts that contain at least one alphabetic character repeated three or more times, and samples 100 rows using `random_state = 42`.

5.2 Detection Rule

Repeated letters are detected using the expression:

```
([A-Za-z])\1{2,}
```

The backreference requires the captured alphabetic character to occur at least two additional times. A threshold of three avoids treating normal English double letters as elongation.

5.3 Normalization Logic

01	Protect	Temporarily replace URLs, #hashtags, and @mentions with placeholders.
02	Detect	Search the remaining text for repeated alphabetic runs.
03	Score	Generate one-repeat and two-repeat candidates and compare English Zipf frequencies with wordfreq.
04	Restore	Preserve UPPERCASE / capitalized form and restore the protected entities.

Table 5. Normalization pipeline implemented in `task2_normalize.py`.

5.4 Protection Scope

The current protection expression covers URLs, hashtags, and mentions. It does not contain a rule for \$cashtags. This difference matters because repeated letters inside financial symbols remain visible to the generic normalization rule and can therefore be shortened.

6. Task 2 - Output Review and Error Analysis

The saved task2_normalized.csv file provides direct before-and-after evidence of both successful corrections and remaining failure modes. Representative cases are summarized below.

Input excerpt	Output excerpt	Assessment
Goood morning Twitter ...	Good morning Twitter ...	Successful reduction
@thyAshes @Tesla Broooooooooo congrats	@thyAshes @Tesla Bro congrats	Successful; mentions preserved
... he is waaaaay overvalued ...	... he is way overvalued ...	Successful reduction
@Jones_Malvin @BaRuvaaa ...	@Jones_Malvin @BaRuvaaa ...	Repeated letters inside mention preserved
\$QQQ \$SQQQ \$TQQQ	\$Q \$SQ \$TQ	Incorrect: cashtags not protected

Table 6. Representative cases from the repository's normalized Task 2 output.

6.1 What Works

- Common expressive elongations can be reduced to shorter lexical forms, such as Goood to Good and waaaaay to way.
- URLs, hashtags, and mentions are masked before normalization and restored afterward.
- The implementation preserves basic casing patterns when returning the selected candidate.

6.2 Limitations

- Cashtags are not explicitly protected. The saved output shows \$QQQ becoming \$Q, \$SQQQ becoming \$SQ, and \$TQQQ becoming \$TQ.
- A frequency score cannot always infer the intended informal word. Slang, multilingual text, usernames, and expressive spellings remain ambiguous.
- Reducing character repetition can remove emphasis that may itself be useful for sentiment or affect analysis.

6.3 Recommended Fix

The clearest improvement is to extend the protected-entity expression with a cashtag rule before normalization:

```
(?<!\w)\$[A-Za-z0-9_]+
```

With this addition, financial identifiers can pass through the same protect-normalize-restore workflow used for URLs, hashtags, and mentions.

7. Conclusion and Recommendations

7.1 Overall Findings

The experiments demonstrate two practical preprocessing issues in social-media Information Retrieval. First, tokenizer choice strongly affects whether platform-specific structures survive as meaningful indexing units. TweetTokenizer performs best under the exact-preservation criterion used here. Second, repeated-character normalization can reduce lexical variation, but only when structured identifiers are protected carefully. The Task 2 output shows that protection rules must cover the full set of identifier types present in the corpus.

7.2 Recommendations

1. Use NLTK TweetTokenizer when the indexing pipeline must retain hashtags, mentions, URLs, and emojis with minimal fragmentation.
2. Apply repeated-character normalization conservatively and retain before/after outputs for inspection.
3. Extend Task 2 protection to cashtags before using the pipeline on finance-related social-media text.
4. Keep the generated CSV and text outputs with the scripts so that every reported figure can be traced back to repository data.

7.3 Repository

Source code, the dataset, intermediate files, and evaluation outputs are available at: <https://github.com/AbhaySingh0303/information-retrieval-assignment>

8. Submission Note

This document has been written from the repository's implemented scripts and saved outputs. The repository does not contain an explicit project-specific AI-usage declaration. If the course requires such a declaration, the student should add an accurate statement describing any AI tools used for coding, debugging, analysis, or report preparation before final submission.